

PostgreSQL VACUUM

Intro

PostgreSQL maintains data consistency using a Multiversion Concurrency Control (MVCC).

This means that each SQL statement sees a snapshot of data (a database version) as it was some time ago, regardless of the current state of the underlying data. This prevents statements from viewing inconsistent data produced by concurrent transactions performing updates on the same data rows, providing transaction isolation for each database session. MVCC, by eschewing the locking methodologies of traditional database systems, minimizes lock contention in order to allow for reasonable performance in multiuser environments.

As a result of this method we have multiple side effects, some of them are:

- Different version of tuples need to be stored (for different transactions)
- Information about which transaction can and can't see a version of a tuple need to be stored
- No longer needed versions (bloat) must be removed from tables and indexes via **VACUUM**
- Various implementation side effects like ID wraparound

VACUUM command

VACUUM is the manual tool to garbage-collect and optionally analyze database objects. It can be used for complete databases or just single tables.

The most important options are

- FULL
- FREEZE
- ANALYZE

Automatic VACUUM

In general, it should not be necessary to run **VACUUM** manually. To archive this PostgreSQL has a mechanism to execute **VACUUM** automatically when needed, as well as throttling it to reduce impact on production.

General cluster wide settings via Chef

The general settings of our PostgreSQL clusters are managed by Chef and can be found in the corresponding roles like `gprd-base-db-postgres.json`.

```
"autovacuum_analyze_scale_factor": "0.005",  
"autovacuum_max_workers": "6",
```

```
"autovacuum_vacuum_cost_delay": "5ms",  
"autovacuum_vacuum_cost_limit": 6000,  
"autovacuum_vacuum_scale_factor": "0.005",  
...  
"log_autovacuum_min_duration": "0",
```

Per table settings

For some workloads custom settings can be beneficial. Think for example of a very large table append only table, which by design does not produce dead tuple, but is expensive to fully scan.

Automated actions and cron jobs

Cron jobs for ANALYZE

Cron jobs to automate ANALYZE were introduced with initial commit for new cron for analyzes.

Cron jobs are defined in `attributes/default.rb` and regularly run `analyze-namespaces-table.sh` and `analyze-issues_notes-table.sh`. At the moment we ANALYZE the following tables `issues`, `notes` and `namespaces`.

Monitoring

Alerts

Challenges

Resource consumption by VACUUM - Optimize PostgreSQL AUTO-VACUUM - 2021

Currently, our AUTOVACUUM setup is really aggressive. During our peak times, we see present a percentage of CPU utilization and IO on the primary database. (links) The goal of this epic is reduce the resource consumption from autovacuum, and keep the database healthy executing the autovacuum routines on the off peak times.

Currently, we are reaching the `autovacuum_freeze_max_age` threshold of 200000000 in less than 3 days on average. Having this configuration so low for our environment forces the execution of AUTOVACUUM TO PREVENT WRAPAROUND in less than 3 days.

Bloat due to infrequent VACUUM

Beside the problem of resource consumption caused by AUTOVACUUM, we also see negative effects by bloated tables and indexes, like 2022-01-21 Web apdex drop.

Strategies and solutions - Optimize PostgreSQL AUTO-VACUUM - 2021

Current State

Currently, our AUTOVACUUM setup is really aggressive. During our peak times, we see present a percentage of CPU utilization and IO on the primary database. (links) The goal of this epic is reduce the resource consumption from autovacuum, and keep the database healthy executing the autovacuum routines on the off peak times.

Currently, we are reaching the `autovacuum_freeze_max_age` threshold of 200000000 in less than 3 days on average.

Having this configuration so low for our environment forces the execution of `AUTOVACUUM TO PREVENT WRAPAROUND` in less than 3 days.

Desired State

We would like to monitor and evaluate if we can optimize the process.

- Create a “mechanism” (I am thinking even a CI pipeline) to execute `VACUUM FREEZE` when the database is idle of the tables that are 80% or 90% of start the `AUTOVACUUM WRAPAROUND`.
- Change the `autovacuum_freeze_max_age` and monitor the impact: Increase `autovacuum_freeze_max_age` from 200000000 to 400000000
- After 2 weeks of analyzing the impact: Increase `autovacuum_freeze_max_age` from 400000000 to 600000000
- After 2 weeks of analyzing the impact: Increase `autovacuum_freeze_max_age` from 600000000 to 800000000
- After 2 weeks of analyzing the impact: Increase `autovacuum_freeze_max_age` from 800000000 to 1000000000
- Change our monitoring to be more efficient

@alexander-sosna: In general, it is recommended not to increase `autovacuum_freeze_max_age`, “If cleaning your house hurts and takes forever, do it more often, not less”. Regarding GitLab’s workload from all around the world, it might be worth a try to shift the `VACUUM` load to a low load time window. We should have short low load windows on a daily basis and longer ones on weekends. Most of the freezing `VACUUM` could be scheduled during these times. Before approaching this we should have confidence in the fact that these windows are sufficient to finish all the work we will delay. We also need an understanding which `autovacuum_freeze_max_age` is needed as a reasonable upper limit. The mechanism to reliably orchestrate `VACUUM` should be in place before any significant increase of `autovacuum_freeze_max_age`, I will move this point up the list.

Major upgrade to PostgreSQL 13

The benchmarked in Benchmark of VACUUM PostgreSQL 12 vs. 13 (btree deduplication) hints us that btree deduplication, introduced in PostgreSQL 13, can help with multiple problems at once.

- Index size
- Index performance
- VACUUM resource consumption

n_dead_tup = 1,000,000

vacuum phase	PG12 (current version)	PG13 (before reindex)	PG13 (with btree deduplication)	PG13 - parallel vacuum (2 parallel workers)
scanning heap	4 min x sec	4 min 18 sec	4 min 51 sec	4 min 16 sec
vacuuming indexes	13 min x sec	13 min 5 sec	10 in 46 sec	3 min 20 sec
vacuuming heap	1 min	52 sec	54 sec	46 sec
total vacuum time	18 min x sec	18 min 16 sec	16 min 31 sec	8 min 24 sec

n_dead_tup = 10,000

vacuum phase	PG12 (current version)	PG13 (before reindex)	PG13 (with btree deduplication)	PG13 - parallel vacuum (2 parallel workers)
scanning heap	5 sec	7 sec	4 sec	5 sec
vacuuming indexes	10 min 39 sec	10 min 28 sec	6 min 11 sec	2 min 18 sec
vacuuming heap	< 1 sec	1 sec	< 1 sec	< 1 sec
total vacuum time	10 min 44 sec	10 min 36 sec	6 min 15 sec	2 min 24 sec

Incidents and issues involving VACUUM

- (Design Document) Configure properly Autovacuum for postgresql

- Review Autovacuum Strategy for all high traffic tables
- Infra review of Autovacuum historical comments and decisions
- Optimize PostgreSQL AUTOVACUUM - 2021
- Lower autovacuuming settings for ci_job_artifacts table
- Benchmark of different VACUUM settings
- Benchmark of VACUUM PostgreSQL 12 vs. 13 (btree deduplication)
- Reduce database index bloat regularly

Other related runbook pages and documents

- Check the status of transaction wraparound Runbook
- pg_xid_wraparound Saturation Alert
- pg_txid_xmin_age Saturation Alert

Literature

- <https://www.postgresql.org/docs/current/mvcc-intro.html>
- <https://www.postgresql.org/docs/current/sql-vacuum.html>
- <https://www.postgresql.org/docs/current/runtime-config-autovacuum.html>